Binary Search & Overflow-Safe Midpoint

Algorithm Precondition & Step Rule

- Input array A[0..n-1] must be sorted; each probe halves the active [low..high] interval.
- If target == A[mid], return mid; if target < A[mid], search left half; else right half.

Avoiding 32-Bit Integer Overflow

- Naive (low + high) // 2 overflows signed 32-bit integers when low + high > INT_MAX.
- Overflow-safe formula: mid = low + (high - low) // 2 stays strictly within array bounds.

Recurrence Relation & Worked Trace

Recurrence Complexity Analysis

- Recurrence: T(n) = T(n/2) + c with constant base case T(1) = c.
- By Master Theorem Case 2 (a=1, b=2, n^(log_2 1)=1): T(n) = Theta(log n) worst/avg case.

Trace on A = [3, 8, 14, 21, 29, 37, 45], target = 37

- Iteration 1: low=0, high=6 -> mid=3 (A[3]=21 < 37) -> update low=4.
- Iteration 2: low=4, high=6 -> mid=5 (A[5]=37 == 37) -> found at index 5 in 2 steps.